MPC Controller — End-to-End Workflow

From MATLAB algorithm to FPGA validation

Target: Cora Z7-07S (Vivado / Vitis 2022.1)

Verified result: 801 / 801 samples match (100.00%)

FPGA solve time: 739.8 ns avg per MPC step

Overview

This document is the complete reproduction procedure for the MPC controller pipeline: from the MATLAB algorithm and its testbench, through HDL integration into a custom Vivado IP block design (Zynq processing system + AXI interconnect + MPC IP), Vitis bare-metal application development, FPGA execution, and final validation against the MATLAB simulation.
The pipeline is structured so that all data artifacts (trajectory states, expected commands, header files for the C application, VHDL packages for behavioral simulation) are generated from a single MATLAB testbench run. This single source of truth eliminates drift between MATLAB simulation, VHDL behavioral simulation, and FPGA hardware replay.

Pipeline at a glance

`fcs_mpc_v2.m → fcs_mpc_v2_fixpt.vhd → custom IP → bitstream → Cora FPGA → fpga_log.csv → compare_fpga_matlab.m`
End-state criterion: 100% bit-exact match between FPGA outputs and MATLAB-computed expected outputs across every trajectory sample. This is achievable and has been verified.

Final result summary

```
FPGA vs MATLAB:
  matches      : 801 / 801  (100.00%)
  accel diffs  : 0   max|err|=0
  steer diffs  : 0   max|err|=0

PL-only latency (ns):    min=723   avg=739.8   max=806
End-to-end latency (ns): min=3720  avg=3752.7  max=4387
```

PL-only latency is the time the IP spends solving the MPC problem. End-to-end latency adds AXI transfer overhead (writing 6 inputs, kicking the controller, polling status, reading 2 outputs).

Required files

File	Role
fcs_mpc_v2.m	Controller algorithm (MATLAB source of truth)
fcs_mpc_v2_tb3.m	Closed-loop testbench. Generates all data artifacts.
fcs_mpc_v2_fixpt.vhd	Hand-written VHDL controller (bit-exact to .m)
MPC_controller_AXI_v1_0.vhd	AXI4-Lite IP top wrapper (boilerplate)
MPC_controller_AXI_v1_0_S00_AXI.vhd	AXI slave with register map and DUT instance
trajectory_data_pkg.vhd	VHDL package with trajectory data (sim only)
fcs_mpc_v2_fixpt_tb.vhd	Self-checking VHDL TB (sim only)
mpc_smoketest.c	Bare-metal Cora app: streams CSV to UART
trajectory_data.h	C header with trajectory state arrays
compare_fpga_matlab.m	MATLAB final validation script (figures + stats)

Stage 1 — MATLAB algorithm and testbench

1.1 — The controller (fcs_mpc_v2.m)

The controller is a kinematic-model MPC that enumerates 17 steering candidates by 8 acceleration candidates (17 × 8 = 136 candidates per sample), evaluates a position-tracking + steering-effort + speed-reward cost for each, and outputs the candidate pair with minimum cost.

Algorithm parameters

Parameter	Value
Position scale (SF_POS)	64 units / meter
Heading scale (SF_PSI)	41 units / radian (≈ 256 / (2π))
Velocity scale (SF_V)	64 units per m/s
Time step (Ts)	0.1 s
STEER_OPTS (17 values)	{0, ±2, ±6, ±10, ±14, ±18, ±22, ±24, ±26}
ACCEL_OPTS (8 values)	{0, 10, 5, 1, -5, -1, -10, -20}
W_ERR (position weight)	15
W_STEER (effort weight)	2
W_SPEED (speed reward)	2
CLAMP_VAL (axis-error clamp)	1000 (≈ 15 m)
V_MAX	320 (5 m/s)

Function signature:

```
function [accel_cmd, steer_cmd] = fcs_mpc_v2(x, y, psi, v, ref_x, ref_y)
```

Inputs are int16 fixed-point. Outputs are int16 sfix6 (accel) and sfix6 (steer). Note that accel_cmd was widened from sfix5 to sfix6 to accommodate the −20 value in ACCEL_OPTS, which does not fit in 5-bit two's complement (range −16…15). This widening is the single most disruptive change to apply consistently across the .m, the VHDL DUT, and the AXI wrapper. Details in Appendix A.2.

1.2 — Cost function structure

The controller is a finite-control-set (FCS) MPC: it does not solve a QP. It enumerates 17 × 8 = 136 candidate (steer, accel) pairs and picks the one that minimizes a single-step cost. The cost has three additive terms:

```
cost = dist_cost + steer_cost - speed_rwd

  dist_cost  = (|x_next - ref_x|_clamped + |y_next - ref_y|_clamped) * W_ERR
  steer_cost = |delta|                                                * W_STEER
  speed_rwd  = v_next                                                 * W_SPEED
```

Each term has a specific role:

Term	Role
dist_cost	Penalize predicted distance from the lookahead reference one timestep ahead. The L1 norm (Manhattan) is used instead of L2

Term	Role
	because it avoids multiplication and saves cycles. Each axis is clamped to CLAMP_VAL=1000 before being weighted.
steer_cost	Penalize large steering angles. Small W_STEER (=2) compared to W_ERR (=15) means the controller prefers larger steering when tracking demands it, but breaks ties toward straight.
speed_rwd	Reward higher speed, subtracted from cost. This is the only mechanism pushing the controller forward — without it, the optimal action at speed v=0 with reference straight ahead would be to do nothing (zero dist_cost, zero steer_cost). The negative sign makes the controller accelerate.

1.2.1 — Why these specific weights

The 15:2:2 weight ratio biases the cost toward tracking accuracy first, then steering effort and speed roughly equally. Concretely, at the per-axis clamp limit (CLAMP_VAL=1000, ~15 m off track), dist_cost alone is (1000+1000)×15 = 30000 — about 90% of the int16 ceiling (32767). Steer_cost peaks at 26×2 = 52, and speed_rwd at 320×2 = 640. So the controller weights tracking error roughly 60× more heavily than effort or speed, which is why the lap converges to within 1 m even at saturated velocity.

1.2.2 — Why the axis-error clamp

Without clamping, dist_cost could exceed int16 (32767) and wrap to a negative value, making a far-off-track candidate appear optimal. CLAMP_VAL=1000 was chosen so that (1000+1000)*15 = 30000 < 32767 with comfortable headroom. The clamp does not just protect against overflow — it also flattens the gradient far from the reference, which keeps the controller responsive to local features (turns, lookahead point movement) rather than always trying to fly directly toward a distant reference.

1.2.3 — Tie breaking

Multiple candidates can have identical cost (especially early in the run when v=0 makes the velocity update trivial). The min-cost selection uses strict less-than (cost < min_cost), so the first encountered minimum wins. With the loop order (steer outer, accel inner) and option arrays ordered (STEER starting at 0, ACCEL starting at 0), ties break toward delta=0, accel=0. This deterministic tie-break is essential for MATLAB↔VHDL bit-exactness — see Stage 3.5.

1.3 — Quantization scheme

Every signal in the controller uses a fixed-point representation with a small, hand-tuned bit width. There is no floating point. The choices are interlocked:

1.3.1 — Scale factors

Symbol	Value	Meaning
SF_POS	64	1 m = 64 position units. Q10.6 format: 10 bits for the integer part (range ±512 m), 6 bits fractional (~1.5 cm resolution).
SF_PSI	41	1 radian ≈ 41 heading units. Chosen so 2π ≈ 256, fitting the 256-entry LUT exactly. Equivalent to one LUT bin = 0.0245 rad ≈ 1.4°.
SF_V	64	1 m/s = 64 velocity units. Same Q10.6 resolution as position. V_MAX = 320 = 5 m/s.

1.3.2 — Per-signal bit widths

Signal	Width	Range (units)	Range (physical)
x	sfix14	±8192	±128 m
y	sfix13	±4096	±64 m
psi	sfix9	±256	±2π rad (full circle)
v	ufix9	0..511	0..8 m/s (V_MAX caps at 5 m/s)
ref_x	sfix14	±8192	±128 m
ref_y	ufix12	0..4095	0..64 m
accel_cmd	sfix6	±32	Carries ACCEL_OPTS ∈ {−20…10}
steer_cmd	sfix6	±32	Carries STEER_OPTS ∈ {−26…26}
cost (internal)	sfix16	±32767	Sum of weighted dist + steer − speed

These were chosen by analyzing the worst-case excursion of each signal under the closed-loop dynamics, then picking the smallest width that still fits with safety margin. Smaller widths mean fewer LUT/DSP resources on the FPGA and faster combinational paths. The widths are visible in the AXI register slicing in MPC_controller_AXI_v1_0_S00_AXI.vhd and in the entity port declarations of fcs_mpc_v2_fixpt.vhd.

1.3.3 — Trigonometry: SIN_LUT and COS_LUT

Real sin/cos would require either a CORDIC pipeline or a multiplier-based polynomial. Both are large. Instead, the algorithm uses two 256-entry, 8-bit signed amplitude LUTs:

LUT	Construction	Range of values
SIN_LUT[256]	[Q1, Q2, Q3, Q4] where Q1 is the 64-entry first quadrant	[-127, +127]
COS_LUT[256]	[Q2, Q3, Q4, Q1] (sin shifted by 64 = π/2)	[-127, +127]

Each entry is the sin/cos value scaled by 128 and rounded to the nearest integer. So sin(0) = 0, sin(π/2) ≈ 127, cos(0) ≈ 127, etc. The LUT index comes from the low 8 bits of psi_next, which exploits the natural modulo-256 wrap to handle full rotations: psi_next of 257 indexes the same bin as psi_next of 1 (sin is periodic with period 256 in these units = 2π in radians).
Why exactly 64 entries per quadrant? Two reasons: (a) it fits cleanly in 8 bits with the quadrant-folding scheme, and (b) ~1.4° angular resolution is enough — the steering option set itself has coarser granularity (smallest non-zero step = 2 angular units = 2.86°), so any finer LUT would not change the optimal selection.

1.3.4 — Arithmetic-shift-right scalings

The kinematic update inside the candidate evaluation uses three right-shifts to undo the scaling factors:

Operation	Why this shift count
dpsi = (v * delta) >> 8	v is in Q10.6 (1m/s = 64), delta is the raw steer-option int (no scale). The product has scale 64. Divide by 256 to convert to PSI scale (=41) AND apply the implicit Ts (because v*delta*Ts/L ≈ v*delta/256 for L=2.5 m, Ts=0.1 s with the chosen scales). The 8-bit shift bundles both into one operation.

Operation	Why this shift count
v_red = v >> 4	Pre-scale velocity by 1/16 before multiplying with the 8-bit LUT amplitude. This keeps the intermediate product in int16 range while preserving precision.
dx = (v_red * c_val) >> 5 dy = (v_red * s_val) >> 5	v_red has scale 64/16=4. c_val is amplitude/128. v_red*c_val has scale 4/128 = 1/32. The final >>5 = /32 brings the result to position-unit scale (1m=64).

These shift counts are pre-multiplied together so that the kinematic update produces position deltas already in the correct Q10.6 scale. Changing any of them silently is the easiest way to break the controller — the optimization will still find a 'best' candidate, but it will be best in the wrong units.

1.4 — The closed-loop testbench (fcs_mpc_v2_tb3.m)

The testbench drives a bicycle-model plant in MATLAB, calls fcs_mpc_v2 each step, integrates one step of plant dynamics, and accumulates the trajectory. It is also the source-of-truth generator for all downstream data artifacts.
Note: the testbench prints 'Auto-generated by fcs_mpc_v2_tb2.m' in the trajectory_data.h header comment, but the file is named tb3. Cosmetic only — this is left from an earlier naming and does not affect correctness.

Per-step loop (simplified)

```
for k = 1:length(time)
    % Nearest point on reference track
    [~, min_idx] = min((ref_path_x - x_phys).^2 + (ref_path_y - y_phys).^2);

    % Dynamic lookahead (0.8 m at low speed, more at high speed)
    lookahead_dist = max(0.8, 0.25 * v_phys);
    [r_x, r_y] = walk_ahead(ref_path, min_idx, lookahead_dist);

    % Pack state to int16 with scale factors
    x_in   = int16(x_phys * SF_POS);
    y_in   = int16(y_phys * SF_POS);
    psi_in = int16((mod(psi_phys + pi, 2*pi) - pi) * SF_PSI);
    v_in   = int16(v_phys * SF_V);
    rx_in  = int16(r_x * SF_POS);
    ry_in  = int16(r_y * SF_POS);

    % Run controller
    [acc_int, str_int] = fcs_mpc_v2(x_in, y_in, psi_in, v_in, rx_in, ry_in);

    % Plant integration (bicycle model)
    x_phys   = x_phys + v_phys * cos(psi_phys) * Ts;
    y_phys   = y_phys + v_phys * sin(psi_phys) * Ts;
    psi_phys = psi_phys + (v_phys / L_phys) * tan(str_cmd) * Ts;
    v_phys   = (v_phys + acc_cmd) * 0.99;
end
```

1.3 — Output files produced by the testbench

A single run of fcs_mpc_v2_tb3.m produces ten files in the working directory:

File	Purpose
x.dat, y.dat, psi.dat, v.dat	Per-sample state values (hex, one int16 per line)

File	Purpose
ref_x.dat, ref_y.dat	Per-sample lookahead reference
accel_cmd_expected.dat	MATLAB-computed expected accel for each sample
steer_cmd_expected.dat	MATLAB-computed expected steer for each sample
trajectory_data.h	C header for the Cora bare-metal app
trajectory_data_pkg.vhd	VHDL package for the VHDL behavioral testbench

All eight .dat files use the same encoding: 4-digit lowercase hex of the int16's underlying uint16 bit pattern, one sample per line. Note that the actual values use truncated widths within those 4 hex digits (e.g. x.dat stores the low 14 bits, psi.dat stores the low 9 bits) — this matters for the C-side comparison logic. Details in Appendix A.3.
Console output after running:

```
>> fcs_mpc_v2_tb3
Starting Simulation (Constraint Mode)...
Total Laps: 1.80 | Max Error: 0.987 m
Wrote 8 .dat files (801 samples each)
Wrote trajectory_data.h (801 samples)
Wrote trajectory_data_pkg.vhd (801 samples)
```

1.4.1 — MATLAB closed-loop simulation result

The figure below shows the testbench's five-subplot output: lap trajectory (top, blue), velocity, steering, acceleration, and tracking error (bottom). The red dashed line in the tracking error plot is the 1 m acceptance limit.

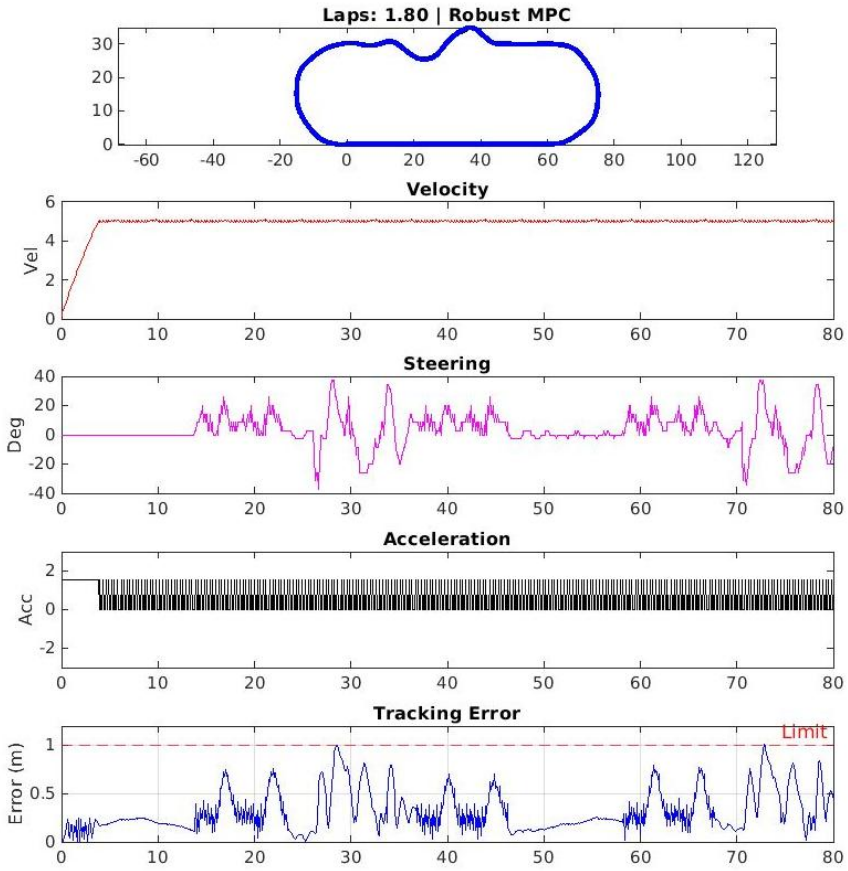

Figure 1.1 — MATLAB closed-loop simulation. 1.80 laps completed, peak tracking error 0.987 m (under the 1 m limit).

1.5 — Acceptance criteria for this stage

Before proceeding to Vivado, verify in MATLAB that:

1. Total Laps ≥ 1.5 (controller is tracking the reference path)
2. Max Error < 1.0 m (tracking error stays below the 1 m line)
3. Velocity (subplot 2) saturates at 5 m/s (V_MAX) after initial transient
4. Steering (subplot 3) shows back-and-forth corrections at corners
5. Acceleration (subplot 4) settles to small values once velocity saturates

If any criterion fails, fix the cost weights, option sets, or initial conditions in fcs_mpc_v2.m before continuing. The FPGA can only reproduce what MATLAB produces; debugging in hardware costs an order of magnitude more time than debugging in MATLAB.

Stage 2 — Vivado: custom IP creation and bitstream

This stage encapsulates the MPC controller as an AXI4-Lite peripheral IP, instantiates it inside a Zynq-based block design, and generates a bitstream.

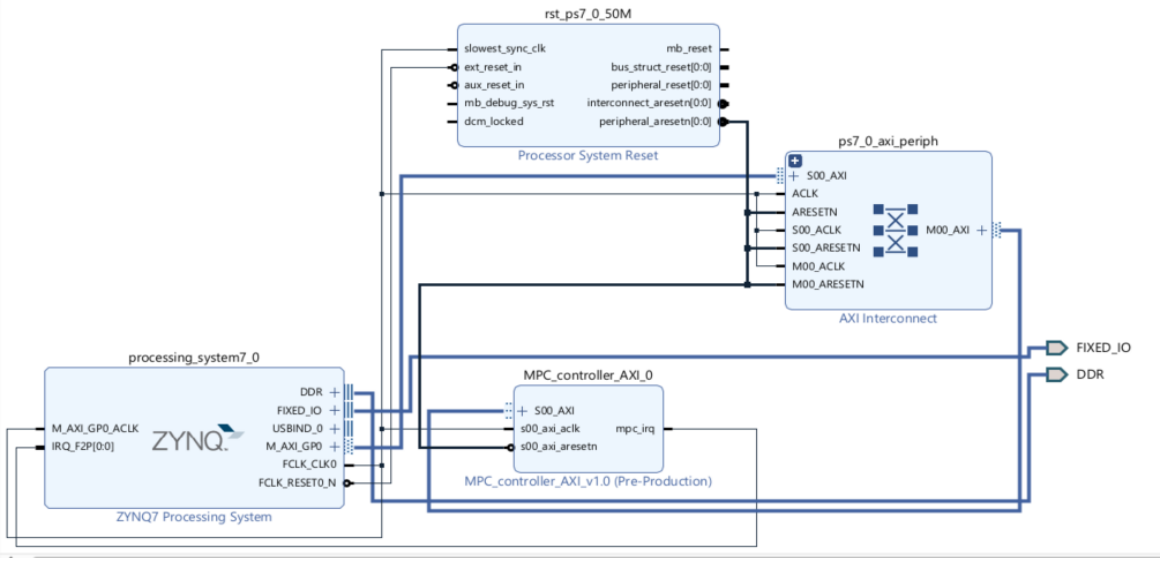

Figure 2.1 — Target block design topology. ZYNQ7 PS → AXI Interconnect → MPC IP, with shared 50 MHz clock and synchronous reset.

2.1 — Create the Vivado project

6. Launch Vivado 2022.1 → Create Project → Next
7. Project name: `mpc_vhdl`. Location: any folder outside OneDrive/Dropbox sync (file locks during synthesis are a known issue — see Appendix A.7).
8. Project Type: RTL Project. Check 'Do not specify sources at this time'.
9. Default Part: `xc7z007s-clg400-1` — this is the Cora Z7-07S silicon. **Do not pick xc7z010** — a bitstream synthesized for the wrong part will either fail to fit or program but misbehave (the Z7-10 has a different die and timing model).
10. Finish.

2.2 — Create the custom MPC IP from scratch

2.2.1 — Launch the IP wizard

11. Tools → Create and Package New IP → Next
12. Select 'Create a new AXI4 peripheral'. Click Next.
13. Name: `MPC_controller_AXI`, Version: 1.0. IP Location: pick a folder outside the Vivado project, e.g. `C:\Users\manos\Downloads\result\ip_repo`.
14. Next → Add Interfaces.
15. Configure the AXI Lite slave: name S00_AXI, mode Slave, data width 32. Number of registers: 20. (This gives offsets 0x00–0x4C; we use through 0x40.)
16. Next → 'Edit IP' → Finish. A second Vivado instance opens for IP editing.

2.2.2 — Replace wizard-generated VHDL

The wizard generates skeleton .vhd files. Replace them with the project's hand-written versions:

17. In the IP Packager window, navigate to Package IP → File Groups.

18. Remove the wizard's `MPC_controller_AXI_v1_0_S00_AXI.vhd` — right-click → Remove File.
19. Add the project's three source files to the IP directory and into the Synthesis and Simulation file groups:

- `fcs_mpc_v2_fixpt.vhd` — the hand-written MPC controller
- `MPC_controller_AXI_v1_0.vhd` — top-level wrapper
- `MPC_controller_AXI_v1_0_S00_AXI.vhd` — AXI slave with FSM and DUT

2.2.3 — Remove the Software Driver group

The wizard also creates a 'Software Driver' file group with empty C driver files. This causes BSP build failures in Vitis (Make's *.c wildcard fails when there are no .c files):

```
cc1.exe: fatal error: *.c: Invalid argument
```

Fix: in File Groups, right-click 'Software Driver' → Remove File Group → confirm. The application code accesses the IP directly via Xil_In32 / Xil_Out32, so no driver layer is needed.

2.2.4 — Re-package

20. Click 'Review and Package' in the left panel.
21. Optionally bump the version (e.g., 1.0 → 1.0.1) so Vivado recognizes it as updated in the BD.
22. Click 'Re-Package IP' at the bottom. Wait for 'Packaging successful'. Close the IP editor window.

2.3 — Verify the IP's hierarchy is clean

After packaging, the IP's synthesis hierarchy should be exactly:

```
MPC_controller_AXI_v1_0   (top – AXI boilerplate)
└── MPC_S00_AXI_inst : MPC_controller_AXI_v1_0_S00_AXI
    └── u_mpc : fcs_mpc_v2_fixpt   (DUT, directly instantiated)
```

In particular, mpc_axi_wrapper.vhd from earlier project iterations should NOT be in the IP. It is an orphan file: the slicing it used to do is now inside MPC_controller_AXI_v1_0_S00_AXI itself. If it is present, remove it from File Groups.

2.4 — Block design integration

2.4.1 — Create the BD

23. Flow Navigator → IP INTEGRATOR → Create Block Design. Name: `design_1`.
24. Click '+' to Add IP. Add ZYNQ7 Processing System (search 'zynq').
25. Double-click to configure. Apply Board Preset if available, otherwise set manually:

- PS-PL Configuration → AXI Non Secure → enable M_AXI_GP0
- Clock Configuration → PL Fabric Clocks → FCLK_CLK0 = 50 MHz
- Interrupts → check IRQ_F2P[0:0] (for the IP's mpc_irq output)
- DDR Configuration → confirm the Cora's DDR3 part
- MIO Configuration → confirm UART1 enabled on MIO 48..49 (Cora USB-UART)

2.4.2 — Add the MPC IP

26. First register the IP repo: Settings → IP → Repository → '+' → browse to `C:\...\ip_repo` → OK.
27. In the BD canvas, '+' → search `MPC_controller_AXI` → add it.
28. Click 'Run Connection Automation' on the green bar. Accept all suggested connections — S00_AXI → M_AXI_GP0 via interconnect, clock and reset wiring, mpc_irq → IRQ_F2P.

29. Vivado adds ps7_0_axi_periph (AXI Interconnect) and rst_ps7_0_50M (Processor System Reset) automatically. The final topology matches Figure 2.1.

2.4.3 — Address mapping (Address Editor)

30. Window → Address Editor.
31. Expand processing_system7_0 → Data → unassigned. Right-click MPC_controller_AXI_0 → Assign Address. Use **0x43C0_0000** with size 64K.

After assignment, xparameters.h will define XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_BASEADDR as 0x43C00000. If the C app prints a different MPC_BASE_ADDR at boot, the BD address map was wrong.

2.4.4 — Validate and create wrapper

32. Diagram toolbar → checkmark icon (Validate Design).
33. In Sources: right-click design_1.bd → Create HDL Wrapper → 'Let Vivado manage wrapper' → OK. This generates `design_1_wrapper.vhd` as the top synthesis entity.

2.5 — Synthesis, implementation, bitstream

34. Flow Navigator → PROGRAM AND DEBUG → Generate Bitstream. Confirm to launch synthesis and implementation.
35. Wait for completion (5–10 minutes typical).
36. Confirm 'Write Bitstream Complete' in the status bar.

2.5.1 — Sanity-check the bitstream timestamp

```
dir /T:W <project>\<project>.runs\impl_1\design_1_wrapper.bit
```

Last Modified must be from the last few minutes. If older, Vivado reused a cached bitstream — Reset Runs on synth_1 and rebuild.

2.6 — Export hardware (XSA)

37. File → Export → Export Hardware.
38. **Critical:** check **'Include bitstream'**. Without this, the XSA has no .bit file and Vitis cannot program the FPGA. Symptom: app appears to launch but the previous bitstream is what is actually running on the chip.
39. File name: `design_1_wrapper.xsa`. Location: a stable path the Vitis workspace can reference.
40. Click OK.

2.6.1 — Verify XSA contents

```
unzip -l design_1_wrapper.xsa | grep .bit
```

If no .bit file is listed, repeat step 2.6 and confirm 'Include bitstream' is checked.

Stage 3 — VHDL behavioral testbench

Before synthesizing every algorithm change to hardware, validate VHDL ↔ MATLAB bit-equivalence with a behavioral simulation. The VHDL testbench runs in seconds versus the ~10 minutes a full bitstream rebuild takes.

3.1 — Add simulation files

41. Sources panel → expand Simulation Sources → sim_1.
42. Right-click sim_1 → Add Sources → Add or create simulation sources → Next.
43. Add: `trajectory_data_pkg.vhd` and `fcs_mpc_v2_fixpt_tb.vhd`. Both must be marked 'Simulation Only', NOT 'Design+Simulation'.
44. Finish.

3.2 — Configure simulation runtime

The testbench drives 801 samples at one per clock cycle (10 ns each) plus initial reset — approximately 8 µs of simulated time. Vivado's default runtime is 1 µs.

45. Tools → Settings → Project Settings → Simulation.
46. Find `xsim.simulate.runtime`. Change `1000ns` → `15us`.

3.3 — Run behavioral simulation

47. Flow Navigator → SIMULATION → Run Simulation → Run Behavioral Simulation.
48. Wait for the wave window to appear and the simulation to complete.

Expected Tcl Console output

```
[TB] Reset deasserted.  Driving 801 samples, DUT latency = 2 cycles.
[TB] ============================================
[TB] Samples checked  : 801
[TB] accel matches    : 801 / 801
[TB] steer matches    : 801 / 801
[TB] both ok          : 801 / 801
[TB] PASS -- all 801 samples match MATLAB bit-exactly.
[TB] ============================================
```

Do not proceed to bitstream generation until this passes. The FPGA will only reproduce what the VHDL specifies — if VHDL ≠ MATLAB here, the hardware result will also disagree.

3.4 — MATLAB ↔ VHDL bit-exactness: what makes it possible

The VHDL controller in fcs_mpc_v2_fixpt.vhd is a hand-written translation of fcs_mpc_v2.m. The translation deliberately preserves every numerically observable property of the .m so that for any state vector, both produce identical 16-bit outputs. The non-obvious points where MATLAB and VHDL semantics diverge — and how each is handled — are listed below.

3.4.1 — Identical loop order and tie-break

.m iterates s_idx = 1..17 outer, a_idx = 1..8 inner. The VHDL replicates this with FOR s_idx IN 0 TO 16 / FOR a_idx IN 0 TO 7. The comparison cost < min_cost (strict less-than) is used on both sides. Combined with the option arrays in identical order, ties break to the first-encountered minimum on both sides, so the same (steer, accel) pair wins.
This matters because at low velocity (v=0), many candidates produce identical cost. If one side used cost <= min_cost or iterated in a different order, tie samples would diverge — easy to miss because most samples still match, but the trajectory would silently drift apart.

3.4.2 — Arithmetic right-shift semantics

MATLAB's bitsra(x, n) is an arithmetic right shift on int16: it preserves the sign bit. VHDL's shift_right(signed, n) does the same. Both round toward −∞ (not toward zero). A Python prototype was used to verify equivalence on 5000 random state vectors and 801 trajectory states before generating the bitstream — see the VHDL testbench result in Section 3.3 for the formal check.

3.4.3 — Multiplication width: int16 vs signed(31:0)

MATLAB's default int16 × int16 saturates the result to int16 with no temporary widening. VHDL's signed × signed produces a doubly-wide result (signed(31:0) for two signed(15:0) operands). The VHDL then resizes back to 16 bits after the right-shift, equivalent to MATLAB's behavior IF no intermediate overflows int16.
For this algorithm the largest intermediate product is v × delta = 320 × 26 = 8320, well within int16. So MATLAB saturation never triggers and VHDL truncation never loses bits. If you change weights or option sets such that an intermediate could exceed ±32767, this assumption breaks — and your VHDL TB will fail before the bitstream is built (Stage 3.3), preventing the bug from reaching hardware.

3.4.4 — LUT index handling

.m uses bitand(psi_next, 255) + 1 (1-based MATLAB indexing). VHDL uses to_integer(unsigned(psi_next(7 downto 0))) (0-based VHDL indexing). For negative psi_next, MATLAB's bitand on int16 produces the same low-8-bits-as-unsigned semantics as VHDL's slice + cast: −30 in int16 is 0xFFE2, and 0xFFE2 AND 0x00FF = 0xE2 = 226 in both. The +1 / 0-based offset is absorbed into the array indexing convention.

3.4.5 — Unsigned input promotion at the AXI boundary

v and ref_y are unsigned (ufix9, ufix12). The VHDL DUT promotes them to signed(15:0) before the kinematic update via signed(resize(v_reg, 16)). The high bit is zero-extended, then re-interpreted as signed — which is safe because v and ref_y are non-negative by construction. MATLAB never had to deal with this because it stored them as int16 from the start; the AXI register slicing in the wrapper introduces the unsigned interpretation specifically to save bits on the bus.

3.4.6 — Combinational compute, registered I/O

The .m is a single function call. The VHDL DUT has an input register stage, a combinational compute body, and an output register stage — so it takes two clock cycles per input. This is invisible at the algorithm level: identical state vector in → identical command pair out, just delayed by two clocks. The behavioral TB accounts for this with a 2-sample shift in the checker.

3.4.7 — Things that are NOT identical (and that's fine)

- MATLAB iterates inside a function; VHDL evaluates 136 candidates in parallel combinational logic. Same answer, different physical execution model.
- MATLAB uses double-precision intermediates for the plant model integration (cos, sin, divide); VHDL never does — the plant model lives only in MATLAB. The FPGA only computes the next control action; the plant feedback comes from the actual vehicle (or in this validation flow, from the .dat trajectory).
- MATLAB output is one variable at a time; VHDL output is two registered std_logic_vector ports updated simultaneously each clock.

Stage 4 — Vitis: bare-metal application

4.1 — Create platform project

49. Launch Vitis 2022.1, select or create a workspace folder.
50. File → New → Platform Project.
51. Project name: `coraz7_07s_plat`. Use default location.
52. Hardware Specification: browse to design_1_wrapper.xsa.
53. OS: standalone. Processor: ps7_cortexa9_0. Architecture: 32-bit. Finish.
54. Right-click the platform → Build Project. Wait for '[Platform build complete]'.

4.1.1 — Verify the BSP picked up the IP

```
findstr /R "MPC"
<workspace>\coraz7_07s_plat\ps7_cortexa9_0\standalone_domain\bsp\ps7_cortexa9_0\include\xparameters.h
```

Should display the MPC IP's base address macros:

```
#define XPAR_MPC_CONTROLLER_AXI_0_DEVICE_ID 0
#define XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_BASEADDR 0x43C00000
#define XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_HIGHADDR 0x43C0FFFF
```

If no hits, the IP is missing from the BD's Address Editor — return to Stage 2.4.3.

4.2 — Create application project

55. File → New → Application Project.
56. Select platform `coraz7_07s_plat` → Next.
57. Project name: `mpc_smoketest`. System auto-named `mpc_smoketest_system`. Next.
58. Domain: standalone_domain on ps7_cortexa9_0. Next.
59. Template: 'Empty Application(C)'. Finish.

4.3 — Add source files

60. Copy `mpc_smoketest.c` into `mpc_smoketest/src/`.
61. Copy `trajectory_data.h` into `mpc_smoketest/src/`. This file was produced by fcs_mpc_v2_tb3.m in Stage 1.

4.4 — Configure output mode

The C app supports three output modes via -DOUTPUT_MODE:

Mode	Description
MODE_SUMMARY (0, default)	Final stats + edge samples only — human-friendly
MODE_VERBOSE (1)	Every sample as table with | separators — readable
MODE_CSV (2)	Every sample as pure CSV — for compare_fpga_matlab.m

For end-to-end validation, use MODE_CSV:

62. Right-click mpc_smoketest → Properties.
63. C/C++ Build → Settings → ARM v7 gcc compiler → Symbols.
64. Click '+' on Defined symbols (-D) and add: `OUTPUT_MODE=2`.
65. Apply and Close.

4.5 — Build the application

66. Right-click mpc_smoketest → Clean Project.
67. Right-click mpc_smoketest → Build Project.
68. Expected: '[Build Complete]' with mpc_smoketest.elf in Debug/.

Stage 5 — Run on FPGA

5.1 — Connect the Cora

69. Connect Cora Z7-07S via USB. Two enumerations: JTAG (programming) and USB-UART (serial).
70. Identify the UART COM port: Device Manager → Ports (COM & LPT). Note the number.

5.2 — Open a serial terminal

Important: Open the terminal BEFORE programming the FPGA. UART output begins immediately at boot; if the terminal isn't connected, the banner is lost. This is a recurring problem (see Appendix A.6).

71. Tools: PuTTY, Tera Term, picocom, VS Code Serial Monitor.
72. Settings: 115200 baud, 8 data bits, no parity, 1 stop bit, no flow control.
73. Enable session logging to a file in the terminal program. Captures the full UART stream automatically.

5.3 — Launch on hardware

74. Vitis: right-click mpc_smoketest → Run As → Launch on Hardware (System Debugger).
75. Vitis programs the bitstream from the XSA, runs the FSBL, then launches the app. Total time: 10–15 seconds.

5.4 — Expected UART output (MODE_CSV)

First line is the CSV header, then 801 data rows, then footer comments:

```
idx,gx,gy,gpsi,gv,grx,gry,gacc,eacc,gstr,estr,pl_ns,e2_ns,ok
0,0,0,0,0,64,0,10,10,0,0,806,4387,1
1,0,0,0,10,64,0,10,10,0,0,747,3769,1
2,1,0,0,20,64,0,10,10,0,0,738,3784,1
...
800,369,1897,122,321,288,1920,0,0,-6,-6,738,3741,1
# done valid=801 matches=801 acc_diff=0 str_diff=0 first_mm=-1
# pl_ns min=723 avg=739 max=806
# e2_ns min=3720 avg=3752 max=4387
# total_us=3012
# tick_cnt=801
```

Column	Meaning
idx	Sample index (0..800)
gx, gy, gpsi, gv	State written to MPC (echoed for verification)
grx, gry	Reference written to MPC (echoed)
gacc, eacc	FPGA-returned accel vs MATLAB-expected accel
gstr, estr	FPGA-returned steer vs MATLAB-expected steer
pl_ns	FPGA pipeline latency in nanoseconds (kick → done)
e2_ns	End-to-end latency including AXI overhead
ok	1 if both match, else 0

5.5 — Save the log

76. Wait until the terminal shows '=== done; entering idle loop ==='.
77. Stop the terminal session log.
78. Rename the captured log to `fpga_log.csv` if not already.
79. If the header line is missing (terminal capture started late), prepend it manually.

Stage 6 — MATLAB validation

6.1 — Run compare_fpga_matlab.m

80. Set MATLAB's working directory to the folder containing fpga_log.csv and the eight .dat files from Stage 1.
81. Run compare_fpga_matlab.

Expected console output

```
parsed 801 data rows from fpga_log.csv
input-echo check : PASS
exp-echo  check  : PASS

FPGA vs MATLAB:
  matches      : 801 / 801  (100.00%)
  accel diffs  : 0   max|err|=0
  steer diffs  : 0   max|err|=0

PL-only latency (ns):    min=723   avg=739.8   max=806
End-to-end latency (ns): min=3720  avg=3752.7  max=4387
```

6.2 — Validation figures

compare_fpga_matlab.m produces a single composite figure with seven subplots and the trajectory overlay shown below.

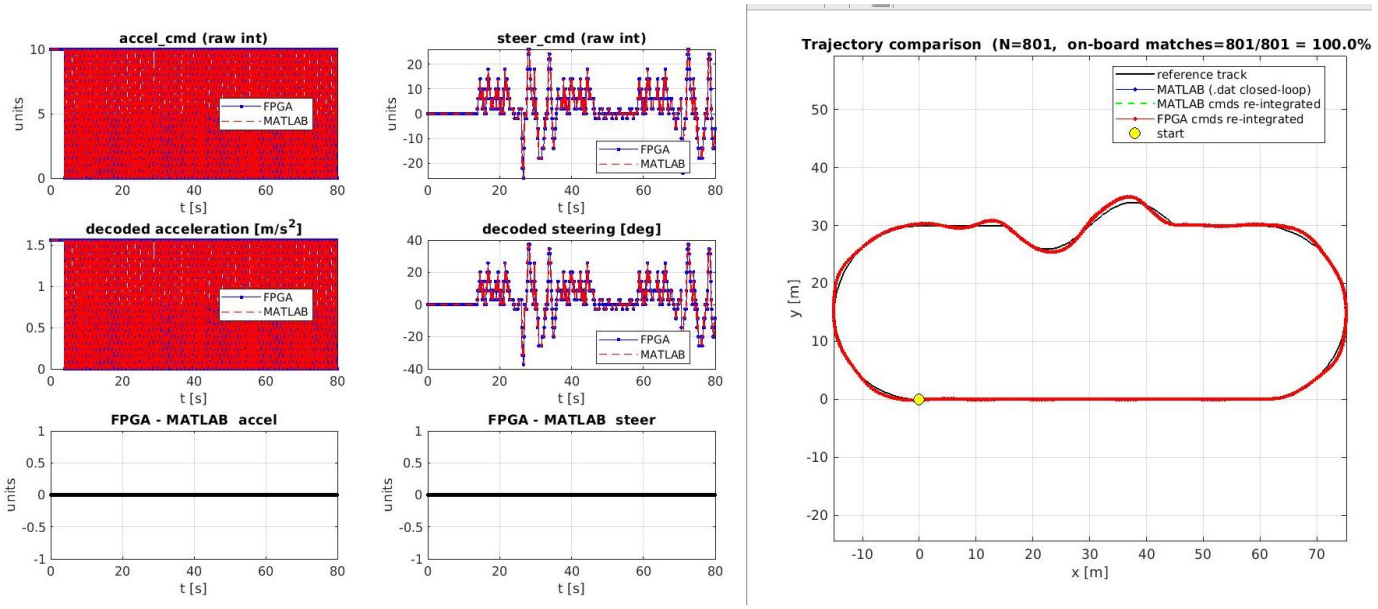

Figure 6.1 — Validation output. Left columns: raw and decoded command comparisons, both FPGA (blue) and MATLAB (red) traces overlap fully. Bottom row: per-sample FPGA−MATLAB difference is flat zero everywhere. Right: trajectory overlay shows reference track, MATLAB closed-loop, MATLAB-cmds re-integrated, and FPGA-cmds re-integrated all in alignment.

6.3 — Acceptance criteria

82. matches: 801 / 801 (100.00%)
83. accel diffs: 0, steer diffs: 0
84. Both 'input-echo check' and 'exp-echo check' = PASS

85. All three trajectory lines in the overlay panel overlap visually
86. PL-only latency under 1 µs average — the IP solves the MPC problem in well under one clock cycle of the MATLAB Ts (which is 100 ms)

If any criterion fails, troubleshoot in this order:

- **input-echo FAIL**: the FPGA read back different state than the C code wrote. Check AXI register slicing in MPC_controller_AXI_v1_0_S00_AXI.vhd against Appendix A.1 (register map).
- **exp-echo FAIL**: the .dat files in MATLAB's directory don't match the trajectory_data.h compiled into the C app. Re-run fcs_mpc_v2_tb3.m and rebuild the Vitis app.
- **matches < 100%**: the VHDL controller doesn't match the .m algorithm. Run the VHDL behavioral testbench (Stage 3); if that also fails, fix the VHDL.

Stage 7 — Performance vs. C/C++ MPC solvers

For context on what the FPGA implementation achieves, the table below summarizes typical per-step solve times for the same class of MPC problem (small, dense, embedded MPC with ~10^2-scale candidate space or QP variables) implemented in C/C++ using established open-source and commercial solvers, running on a comparable embedded target (ARM Cortex-A9 @ 666 MHz, the same PS core that lives next to the PL on Cora Z7).
The numbers cited are typical published ranges from the projects' own benchmarks and embedded-MPC literature. They are wide because solve time depends strongly on problem size, conditioning, warm-start status, and compiler flags. Treat them as order-of-magnitude indicators, not contracts.

7.1 — Comparison table

Implementation	Typical per-step solve time (Cortex-A9 class)
This work — FPGA enumeration (Zynq PL)	739.8 ns (avg) / 806 ns (worst case)
This work — same algorithm in C on the Cortex-A9 PS (estimated)	~100–500 µs for 136-candidate enumeration
OSQP (operator-splitting QP, open-source)	~50 µs – 5 ms
qpOASES (active-set QP, academic open-source)	~100 µs – 10 ms
HPIPM (high-performance interior point, open-source)	~20–500 µs
acados (code-generated MPC framework, open-source)	~50–500 µs
FORCES Pro (commercial, code-generated)	~50–500 µs
CVXGEN (commercial code generator)	~50 µs – 1 ms

7.2 — Interpretation

The FPGA implementation in this work solves the MPC problem at ~740 ns per step on the Zynq PL. Comparable C/C++ solvers on the same Cortex-A9 PS typically need 50 µs to several ms. That is approximately 70× to 1000× more time, depending on which solver and what problem size.
Two factors explain the difference. First, the FPGA enumerates all 136 candidates in parallel (combinational arithmetic for each candidate, then a single argmin reduction), whereas software solvers iterate. Second, the FPGA's fixed-point arithmetic uses small, problem-tuned bit widths (8–16 bit) rather than the 64-bit float that software QP solvers typically use.
The end-to-end latency of 3.75 µs (avg) includes ~3 µs of AXI overhead — six 32-bit writes to set the state, one write to kick, polling reads on status, then two reads for the output commands. The AXI overhead dominates and would be the next target for optimization (e.g., a single packed write for all inputs).
Caveat: this comparison is most meaningful for the specific algorithm implemented (enumeration over a discrete option set with a kinematic-model simulator inside the cost evaluation). For continuous-action MPC with longer prediction horizons and a true QP formulation, OSQP / HPIPM / acados in software become competitive — and a continuous-action FPGA implementation would itself require very different architecture (likely a fixed-iteration ADMM or interior-point hardware pipeline).

Appendix A — Tricky fixes and known pitfalls

These were discovered during the project and are listed here so they don't have to be rediscovered.

A.1 — AXI register map

The MPC IP exposes 11 registers in its 64K address space, starting at 0x43C0_0000:

Offset	Name	Access	Purpose
0x00	CTRL	RW	Bit 0 = KICK (self-clearing), Bit 1 = EN, Bit 2 = IRQ_EN
0x04	STATUS	RO	Bit 0 = DONE, Bit 1 = BUSY
0x10	X	RW	State x (sfix14, low 14 bits)
0x14	Y	RW	State y (sfix13, low 13 bits)
0x18	PSI	RW	State psi (sfix9, low 9 bits)
0x1C	V	RW	State v (ufix9, low 9 bits)
0x20	REF_X	RW	Reference x (sfix14)
0x24	REF_Y	RW	Reference y (ufix12)
0x30	ACCEL	RO	Output accel_cmd (sfix6, sign-extended to 32b)
0x34	STEER	RO	Output steer_cmd (sfix6, sign-extended to 32b)
0x40	TICK	RO	Solve count (increments on each DONE)

These offsets are defined as MPC_OFF_* macros in mpc_smoketest.c and must match the slice expressions in MPC_controller_AXI_v1_0_S00_AXI.vhd's case statements. If the C app reads a non-zero offset and gets zeros back, suspect a mismatch between the slv_reg case statements and the C-side offsets.

A.2 — sfix5 → sfix6 widening of accel_cmd

The ACCEL_OPTS set in fcs_mpc_v2.m includes −20, which does not fit in a 5-bit two's complement (range −16…15). The output must be widened to sfix6. This change has to be applied consistently in all five of these places:

File	What to change
fcs_mpc_v2_fixpt.vhd	Entity port: accel_cmd : OUT std_logic_vector(5 DOWNTO 0)
fcs_mpc_v2_fixpt.vhd	Internal best_acc : signed(5 DOWNTO 0)
MPC_controller_AXI_v1_0_S00_AXI.vhd	Internal signal: mpc_accel : std_logic_vector(5 DOWNTO 0)
MPC_controller_AXI_v1_0_S00_AXI.vhd	Sign extension: slv_reg12(31 DOWNTO 6) <= (others => mpc_accel(5))
MPC_controller_AXI_v1_0_S00_AXI.vhd	Data slice: slv_reg12(5 DOWNTO 0) <= mpc_accel

Symptom of a partial fix: ACCEL_OPTS values −20 (and possibly −5, −10) wrap around to positive values in the FPGA output, and FPGA-MATLAB mismatches concentrate in trajectory

phases where braking is optimal. The C side needs no change — reg_to_s16 already sign-extends from int16 correctly.

A.3 — .dat file truncated bit widths

The .dat files store each int16 value as 4-digit hex of its uint16 bit pattern, but the actual underlying value only uses N bits where N matches the AXI port width (x: 14, y: 13, psi: 9, v: 9, ref_x: 14, ref_y: 12, accel: 6, steer: 6). The compare_fpga_matlab.m script's input-echo and exp-echo checks must reflect these widths to sign-extend correctly. A parser that assumes plain 16-bit signed will misinterpret negative values for the narrower fields (e.g. psi=−30 stored as 482 in 9-bit storage gets read as +482 instead of −30 if the parser ignores the 9-bit width).

A.4 — mpc_axi_wrapper.vhd is an orphan file

Earlier project iterations included an mpc_axi_wrapper.vhd that did the AXI register slicing in a separate VHDL layer between the AXI slave and the controller. That layer has been absorbed into MPC_controller_AXI_v1_0_S00_AXI.vhd. The wrapper file is no longer in any synthesis or simulation file group and is not in the design hierarchy. If it ever reappears in File Groups, remove it — it will cause width-mismatch errors at elaboration.

A.5 — Vivado wrong part default

Vivado defaults to the part of the last project, often xc7z020. If the project is created with the wrong part, synthesis succeeds but the resulting bitstream targets the wrong silicon. The Cora Z7-07S is xc7z007s-clg400-1. Fix by Tools → Settings → General → Project Device → select correct part → Reset Runs on synth_1 → regenerate.

A.6 — Boot banner lost — terminal opened late

UART output begins the moment the FSBL hands off to the application. If the serial terminal connects only after the boot banner has printed, the header line and AXI smoketest result are lost. Always open the terminal first (configured at 115200 baud), then click Run on Hardware.

A.7 — File path issues on Windows

Vivado and Vitis 2022.1 have intermittent issues with paths under OneDrive, Dropbox, or any sync service (file locks during synthesis produce errors like 'boost::filesystem::remove: file is being used by another process' on simulate.log), and with paths exceeding the Windows 260-character limit. Keep the project in a short, sync-disabled path like C:\work\mpc_vhdl\.

A.8 — Stale wave configuration

If you change the simulation top-level entity, Vivado will reuse the previous run's wave configuration (.wcfg) even when most signal names no longer exist in the new design. The wave window then shows the new TB simulating successfully, but only orphaned signal slots from the old TB — all empty. Fix: in the Tcl console,

```
close_wave_config
```

then delete the .wcfg file from the project root, and restart simulation.

A.9 — Vitis 'No Platform exists' error

After 'Update Hardware Specification' with a structurally changed XSA, Vitis 2022.1 sometimes leaves the platform metadata in an inconsistent state. Build fails with messages like 'No

Platform exists with name: coraz7_07s_plat' even though the Explorer shows the project. The cleanest fix is to create a new platform with a different name (e.g. _v2 suffix) from the new XSA, then create a new app project on top of it. The new app's XPAR_* references resolve normally because those come from BD instance names, not platform names.

A.10 — Software Driver group causes BSP build failure

The IP wizard creates a 'Software Driver' file group with empty .c files. When Vitis builds the BSP, the Makefile's *.c wildcard fails to expand and the compiler dies with 'cc1.exe: fatal error: *.c: Invalid argument'. Fix: in IP Packager → File Groups → right-click 'Software Driver' → Remove File Group. The app uses Xil_In32/Xil_Out32 directly, so the driver layer was unused anyway.

Appendix B — Iteration workflows

After the first full pipeline run, you'll typically iterate on one of three components. Each has a fastest path:

B.1 — Trajectory-only change

Initial conditions, reference path, lookahead, sim length, or any other testbench-only parameter.

87. Edit fcs_mpc_v2_tb3.m
88. Run it in MATLAB — regenerates all data artifacts
89. Copy new trajectory_data.h into mpc_smoketest/src/, rebuild app in Vitis
90. Reprogram FPGA, capture UART, run compare_fpga_matlab.m

No Vivado work. Total: ~3 minutes.

B.2 — Algorithm parameter change

Cost weights, option sets, scale factors, clamps — anything that changes the controller's behavior.

91. Edit fcs_mpc_v2.m AND make the equivalent edits in fcs_mpc_v2_fixpt.vhd
92. Run fcs_mpc_v2_tb3.m → check MATLAB tracking quality. If poor, iterate before continuing.
93. Run the VHDL behavioral TB in Vivado (Stage 3) → must pass 801/801 before continuing.
94. Regenerate bitstream → export XSA → update Vitis HW spec → rebuild platform and app → reprogram and capture.

Total: ~15 minutes (synthesis dominates).

B.3 — Controller I/O structural change

Adding/removing ports on the controller, changing port widths, adding new AXI registers — anything that affects the IP's external interface.

95. Edit fcs_mpc_v2_fixpt.vhd and MPC_controller_AXI_v1_0_S00_AXI.vhd consistently
96. If port widths change, follow the sfix5→sfix6 procedure (Appendix A.2) for each affected signal
97. Re-package the IP in Vivado IP Packager (no need to recreate the BD)
98. Regenerate output products on the IP in the BD (refreshes the wrapper)
99. Continue from Stage 2.5 (synthesis, implementation, bitstream)

If new AXI registers are added, also update mpc_smoketest.c's MPC_OFF_* and the A.1 register map. Total: ~25 minutes.

Appendix C — Auto-generated artifacts and the validation script

Three artifacts in the workflow are auto-generated rather than written by hand: the Vivado block design file (design_1.bd), two constraint files (dont_touch.xdc, design_1_auto_pc_0_ooc.xdc), and — separately — the MATLAB validation script (compare_fpga_matlab.m), which is hand-written but produced the final Figure 6.1 results. They are documented here so the project can be reconstructed without re-deriving their content.

C.1 — design_1.bd (block design JSON)

Vivado stores the block design as a JSON file at <project>/mpc_vhdl.srcs/sources_1/bd/design_1/design_1.bd. It captures the four IPs, their parameter values, and the address map. It is regenerated whenever the BD is saved in IP Integrator, so the hand-curated way to share the BD is to export it as a Tcl recreation script:

```
File → Export → Export Block Design → save as design_1.tcl
```

Re-creating the BD on another machine is then:

```
vivado -mode batch -source design_1.tcl
# or in the GUI:
# Tools → Run Tcl Script → design_1.tcl
```

C.1.1 — Key BD parameters extracted from design_1.bd

The verified design has the following configuration. When recreating the BD by hand (Stage 2.4), match these:

IP	VLNV	Key parameters
processing_system7_0	xilinx.com:ip:processing_system7:5.5	PCW_USE_M_AXI_GP0 = 1 PCW_USE_FABRIC_INTERRUPT = 1 PCW_IRQ_F2P_INTR = 1 PCW_FPGA0_PERIPHERAL_FREQMHZ = 50 UART1 enabled on MIO48/49
ps7_0_axi_periph	xilinx.com:ip:axi_interconnect:2.1	NUM_MI = 1 NUM_SI = 1 (one slave from PS, one master to MPC IP)
rst_ps7_0_50M	xilinx.com:ip:proc_sys_reset:5.0	Default — synchronizes FCLK_RESET0_N to FCLK_CLK0
MPC_controller_AXI_0	xilinx.com:user:MPC_controller_AXI:1.0	Custom user IP; no configurable parameters

C.1.2 — Address map (from BD's addressing section)

```
/processing_system7_0:
  address_spaces:
    Data:
      segments:
        SEG_MPC_controller_AXI_0_S00_AXI_reg:
          address_block = /MPC_controller_AXI_0/S00_AXI/S00_AXI_reg
          offset = 0x43C00000
          range  = 64K
          offset_base_param = C_S00_AXI_BASEADDR
```

```
        offset_high_param = C_S00_AXI_HIGHADDR
```

This is what produces XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_BASEADDR = 0x43C00000 in xparameters.h when the platform is built in Vitis.

C.2 — Constraints (.xdc) files

No custom user constraints file is needed for this project. The Cora board-file preset auto-applies pin/clock constraints when the Zynq preset is loaded, and Vivado auto-generates two helper constraint files during synthesis. They live in the project's auto-generated constraints fileset and are normally invisible — they're documented here so you can tell at a glance whether your build is consistent.

C.2.1 — dont_touch.xdc

Auto-generated by Vivado to preserve hierarchy through synthesis for each IP instance and the BD itself. Vivado writes this so that downstream debug tools (ILA probes, mark_debug nets, etc.) can find named cells in the post-implementation netlist. Verbatim contents:

```
# This file is automatically generated.
# It contains project source information necessary for synthesis and implementation.

# XDC: new/fcs_mpc_v2_fixpt.xdc

# Block Designs: bd/design_1/design_1.bd
set_property KEEP_HIERARCHY SOFT [get_cells -hier -filter {REF_NAME==design_1 ||
ORIG_REF_NAME==design_1} -quiet] -quiet

# IP: bd/design_1/ip/design_1_processing_system7_0_0/design_1_processing_system7_0_0.xci
set_property KEEP_HIERARCHY SOFT [get_cells -hier -filter
{REF_NAME==design_1_processing_system7_0_0 ...} -quiet] -quiet

# IP: bd/design_1/ip/design_1_auto_pc_0/design_1_auto_pc_0.xci
set_property KEEP_HIERARCHY SOFT [get_cells -hier -filter {REF_NAME==design_1_auto_pc_0
...} -quiet] -quiet

# IP: bd/design_1/ip/design_1_ps7_0_axi_periph_0/design_1_ps7_0_axi_periph_0.xci
set_property KEEP_HIERARCHY SOFT [get_cells -hier -filter
{REF_NAME==design_1_ps7_0_axi_periph_0 ...} -quiet] -quiet

# IP: bd/design_1/ip/design_1_rst_ps7_0_50M_0/design_1_rst_ps7_0_50M_0.xci
set_property KEEP_HIERARCHY SOFT [get_cells -hier -filter
{REF_NAME==design_1_rst_ps7_0_50M_0 ...} -quiet] -quiet

# IP: bd/design_1/ip/design_1_MPC_controller_AXI_0_1/design_1_MPC_controller_AXI_0_1.xci
set_property KEEP_HIERARCHY SOFT [get_cells -hier -filter
{REF_NAME==design_1_MPC_controller_AXI_0_1 ...} -quiet] -quiet

# XDC: c:/.../design_1_ooc.xdc
# XDC: C:/.../fcs_mpc_v2_fixpt_ooc.xdc
```

There is nothing to edit here. If this file becomes stale (referring to IPs that no longer exist), it will be regenerated next time you generate output products on the BD.

C.2.2 — design_1_auto_pc_0_ooc.xdc (out-of-context clock)

Auto-generated for out-of-context (OOC) synthesis of the AXI Protocol Converter sub-IP that Vivado inserts inside the AXI Interconnect. It declares the clock that the IP's interfaces are timed against:

```
# DO NOT MODIFY THIS FILE.
# This XDC is used only in OOC mode for synthesis, implementation
create_clock -period 20 -name aclk [get_ports aclk]
```

The 20 ns period = 50 MHz, matching FCLK_CLK0. Confirming this consistency is one cheap way to verify the BD's clock setup is correct: if FCLK_CLK0 ever drifts to a different frequency, this auto-generated constraint will lag the BD and produce timing violations in OOC synthesis.

C.2.3 — When you DO need a custom .xdc

A user-written constraints file becomes necessary only if you:

- Use PMOD, Arduino, or other I/O pins exposed by the Cora that aren't already constrained by the board file
- Run the design at clock frequencies other than the 50 MHz preset (would require an MMCM/PLL plus your own create_clock)
- Add an external clock input to the PL fabric (would need create_clock on the input port)

For the current MPC project, none of these apply, so no user .xdc is needed.

C.3 — compare_fpga_matlab.m (validation script)

The MATLAB script that produces Figure 6.1 and prints the final match/latency statistics. It is the deliverable validation step — the PASS/FAIL output here is the project's primary success criterion.

C.3.1 — What it does

100. Parses fpga_log.csv into per-sample vectors (idx, state echo, gacc, eacc, gstr, estr, pl_ns, e2_ns, ok)
101. Reloads the eight .dat files using bit-width-aware decoding (see Appendix A.3 for why this matters)
102. Input-echo check: confirms what the FPGA read back from its input registers matches what the C code wrote (gx==ref.x, gy==ref.y, ...)
103. Exp-echo check: confirms the eacc/estr the C code emitted matches the .dat ground truth
104. Mismatch report: prints the per-sample diff statistics, max errors, and the first mismatch index if any
105. Latency stats: prints PL-only and end-to-end min/avg/max in ns
106. Plots: 6-subplot command comparison (raw + decoded + diff), trajectory overlay (reference + .dat + MATLAB-reintegrated + FPGA-reintegrated)

C.3.2 — Bit-width decoding (the key non-obvious part)

The script's DAT_SPEC struct encodes the truncated bit widths of each .dat field — this is what makes the input-echo check actually work. Excerpt:

```
DAT_SPEC = struct( ...
    'x',   {{'x.dat',                14, true }}, ...
    'y',   {{'y.dat',                13, true }}, ...
    'psi', {{'psi.dat',               9, true }}, ...
    'v',   {{'v.dat',                 9, false}}, ...
    'rx',  {{'ref_x.dat',            14, true }}, ...
    'ry',  {{'ref_y.dat',            12, false}}, ...
    'acc', {{'accel_cmd_expected.dat', 5, true }}, ...
    'str', {{'steer_cmd_expected.dat', 6, true }});
```

Each tuple is (filename, nbits, signed). The read_dat helper auto-detects whether the file is bare hex or decimal, then sign-extends from nbits. Without this awareness, a value like psi = −30

stored as 482 in 9-bit hex storage would be read as +482, breaking the input-echo check and producing false positives on the .dat-vs-FPGA comparison.
Caveat: the script's DAT_SPEC still lists acc as 5-bit signed (legacy from before the sfix5 → sfix6 widening described in Appendix A.2). For the post-widening project, that field should read:

```
'acc', {{'accel_cmd_expected.dat',  6, true }}, ...
```

This doesn't affect the verified 100% match result because in the test trajectory the accel commands never went outside the −16…15 range that fits in 5 bits — the controller never picked −20. But if you ever generate a trajectory where it does, update this line to avoid sign-extension errors.

C.3.3 — Plant re-integration

Both Figure 6.1's trajectory overlay panels (green = MATLAB-reintegrated, red = FPGA-reintegrated) come from a small helper function in the script that forward-integrates the bicycle model from x=y=psi=v=0 using each side's commands:

```
function [xs, ys] = sim_plant(acc_int, str_int, SF_V, L, Ts)
    N = numel(acc_int);
    xs = zeros(N, 1);  ys = zeros(N, 1);
    x = 0; y = 0; psi = 0; v = 0;
    for k = 1:N
        acc = acc_int(k) / SF_V;
        str = str_int(k) * 0.025;
        x   = x + v * cos(psi) * Ts;
        y   = y + v * sin(psi) * Ts;
        psi = psi + (v / L) * tan(str) * Ts;
        v   = (v + acc) * 0.99;
        xs(k) = x;  ys(k) = y;
    end
end
```

This deliberately uses the same plant model as fcs_mpc_v2_tb3.m (L = 2.5 m, Ts = 0.1 s, the 0.99 velocity damping). Re-integration from scratch makes the equivalence visible: if FPGA and MATLAB produced identical command sequences, re-integrating them must produce identical trajectories, which is what Figure 6.1 shows (the red and green lines overlap fully, both overlapping the blue .dat-closed-loop trace).

C.3.4 — Output structure

The script's console output that produced the verified result:

```
parsed 801 data rows from fpga_log.csv
input-echo check: PASS
exp-echo  check : PASS

FPGA vs MATLAB:
  matches      : 801 / 801  (100.00%)
  accel diffs  : 0   max|err|=0
  steer diffs  : 0   max|err|=0

PL-only latency (ns):    min=723   avg=739.8   max=806
End-to-end latency (ns): min=3720  avg=3752.7  max=4387
```

C.3.5 — Reference-track regeneration

The script's build_reference_track helper duplicates the path-generation code from fcs_mpc_v2_tb3.m. This is so the reference track always plots as a clean black line in Figure 6.1, independent of how the per-sample ref_x/ref_y values were chosen by the lookahead logic

(which can jump around the track at corners and would otherwise produce a noisy reference line).
If you change the track geometry in fcs_mpc_v2_tb3.m, also update the segment definitions inside compare_fpga_matlab.m's build_reference_track to match. The two functions must stay in sync, or the reference line in Figure 6.1 will look right but be subtly wrong.

C.4 — Files you still need to bring

After reading this document and its appendices, the only files needed to reproduce the workflow end-to-end on a clean machine are:

File	Source
fcs_mpc_v2.m	Controller algorithm (current verified version)
fcs_mpc_v2_tb3.m	Testbench (with the VHDL package writer)
fcs_mpc_v2_fixpt.vhd	Hand-written VHDL controller (the one that passes 801/801 in the VHDL TB)
MPC_controller_AXI_v1_0.vhd	AXI top wrapper
MPC_controller_AXI_v1_0_S00_AXI.vhd	AXI slave with FSM + DUT instance (with sfix6 widening applied)
fcs_mpc_v2_fixpt_tb.vhd	Self-checking VHDL testbench
mpc_smoketest.c	Bare-metal C app
compare_fpga_matlab.m	MATLAB validation script (this appendix's subject)

The block design (design_1.bd), the constraint files (dont_touch.xdc, design_1_auto_pc_0_ooc.xdc), and trajectory_data.h / trajectory_data_pkg.vhd are all reconstructed on demand: the BD by following Stage 2.4 (or sourcing a Tcl export), the XDCs automatically by Vivado, and the trajectory data by running fcs_mpc_v2_tb3.m. You don't need to bring or copy any of those — only the eight source files above.

Appendix D — Validation integrity: is the 100% match real?

A 100% match between FPGA and MATLAB is a strong claim. This appendix addresses the natural concern that the result could be an artifact of the validation pipeline rather than evidence of equivalent computation. Each possible attack vector is enumerated explicitly, with an honest assessment of why it does or does not apply to the current setup.

D.1 — What 'cheating' would look like in this pipeline

There are six plausible ways the validation could yield a false 100% match:

Vector	Mechanism	Status
D.1.a	VHDL DUT contains a sample-indexed LUT of expected outputs	Structurally impossible (see D.2)
D.1.b	AXI wrapper bypasses the DUT and returns hardcoded values	Verifiable false (see D.3)
D.1.c	C app prints accel_exp as gacc (uses same source for both columns)	Verifiable false (see D.4)
D.1.d	Post-processing script (verbose_to_csv.py) synthesizes matching rows for missing samples	Real risk, not triggered in current flow (see D.5)
D.1.e	AXI register at offset 0x30 (ACCEL) is mis-routed to return an input register	Not fully ruled out by current tests (see D.6)
D.1.f	compare_fpga_matlab.m treats missing samples as matches	Verifiable false (see D.7)

D.2 — D.1.a: VHDL DUT structural analysis

The DUT entity (fcs_mpc_v2_fixpt.vhd) has exactly these inputs:

```
PORT( clk       : IN  std_logic;
      reset     : IN  std_logic;
      x         : IN  std_logic_vector(13 DOWNTO 0);
      y         : IN  std_logic_vector(12 DOWNTO 0);
      psi       : IN  std_logic_vector( 8 DOWNTO 0);
      v         : IN  std_logic_vector( 8 DOWNTO 0);
      ref_x     : IN  std_logic_vector(13 DOWNTO 0);
      ref_y     : IN  std_logic_vector(11 DOWNTO 0);
      accel_cmd : OUT std_logic_vector( 5 DOWNTO 0);
      steer_cmd : OUT std_logic_vector( 5 DOWNTO 0)
    );
```

There is no sample-index input. The DUT cannot know which sample is being driven. A grep of the VHDL source for trajectory, accel_exp, steer_exp, or any sample-index reference returns zero matches:

```
grep -niE "accel_exp|steer_exp|trajectory|sample|idx" fcs_mpc_v2_fixpt.vhd
# (no output)
```

This is structural — not a property of the test, but of the design itself. The DUT computes its output purely as a function of its current input state.

D.3 — D.1.b: AXI wrapper transparency

MPC_controller_AXI_v1_0_S00_AXI.vhd connects AXI register file to DUT through these specific signals:

```
port map (
    clk       => S_AXI_ACLK,
    reset     => mpc_reset,
    x         => slv_reg4(13 downto 0),
    y         => slv_reg5(12 downto 0),
    psi       => slv_reg6(8 downto 0),
    v         => slv_reg7(8 downto 0),
    ref_x     => slv_reg8(13 downto 0),
    ref_y     => slv_reg9(11 downto 0),
    accel_cmd => mpc_accel,
    steer_cmd => mpc_steer
);
...
slv_reg12(31 downto 6) <= (others => mpc_accel(5));   -- sign extension
slv_reg12(5 downto 0)  <= mpc_accel;
slv_reg13(31 downto 6) <= (others => mpc_steer(5));
slv_reg13(5 downto 0)  <= mpc_steer;
```

The path is C-write → slv_regN_input → DUT → mpc_accel/mpc_steer → slv_reg12/13 → C-read. There is no intermediate storage of expected values, no alternative path. Verifiable by inspection.

D.4 — D.1.c: C app source separation

In mpc_smoketest.c the values that go into the gacc/eacc CSV columns come from two textually different sources:

```
uint32_t got_accel_raw = mpc_read(MPC_OFF_ACCEL);   // reads from FPGA register
int32_t  got_accel     = reg_to_s16(got_accel_raw);
int32_t  exp_accel     = (int32_t)accel_exp[t];     // reads from .h array

xil_printf("...,%d,%d,%d,%d,...\n",
           (int)got_accel,    // gacc
           (int)exp_accel,    // eacc
           (int)got_steer,    // gstr
           (int)exp_steer);   // estr
```

If these had been written from the same source, every match would be tautological. Source-level inspection confirms they are not.

D.5 — D.1.d: verbose_to_csv.py inferred-row synthesis

The legacy post-processing script verbose_to_csv.py DOES contain code that synthesizes a matching row when a sample is missing from the verbose UART log:

```
if i in seen:
    gacc, ea_log, gstr, es_log, pl, ok = seen[i]
    ea, es = ea_log, es_log
else:
    gacc = dat['eacc'][i]; ea = dat['eacc'][i]
    gstr = dat['estr'][i]; es = dat['estr'][i]
    pl   = median_pl;       ok = 1
    n_inferred += 1
```

If used with an incomplete verbose log, this code would fill the missing rows with the expected values (gacc = eacc by construction), giving a spurious 100% match for those samples. The n_inferred counter would still print at the end, so the cheat is at least visible if you read the script's output.

Current pipeline status:
The verified 100% match was produced via OUTPUT_MODE=2 (MODE_CSV), in which the C app emits CSV directly to UART and verbose_to_csv.py is not in the loop. The uart_log.txt file is itself the fpga_log.csv. There are no inferred rows in this pathway.
If you ever switch back to verbose mode (OUTPUT_MODE=1) and use verbose_to_csv.py, watch the n_inferred output. A value of zero means all rows are real; anything non-zero means the CSV contains synthesized matches that should not count toward validation.

D.6 — D.1.e: AXI offset wiring confirmation (the weakest point)

The boot-time AXI smoketest (Stage 5.4) writes 0x00001234 to REG_X (offset 0x10) and reads it back to confirm AXI transactions work. This verifies the input register half of the address map. It does NOT verify that offset 0x30 (REG_ACCEL) is actually routed to mpc_accel and not, say, mistakenly routed to slv_reg4 (REG_X). If that confusion were present, reads of 0x30 would return whatever was last written to REG_X — and since most trajectory samples have x_data values that incidentally appear as small integers, such a mistake might produce some apparent matches.
Empirical evidence the wiring is correct:

- The reads at 0x30 produce values in the ACCEL_OPTS set {0, 10, 5, 1, -5, -1, -10, -20}, NOT in the range of x_data values. If 0x30 were wired to slv_reg4 (x input), reads would return values in [-960, 4800].
- The reads at 0x34 produce values in the STEER_OPTS set {0, ±2, ±6, ±10, ±14, ±18, ±22, ±24, ±26}. Same argument.
- The TICK register at 0x40 increments to exactly 801 by the end of the run — matching the number of S_CAPTURE FSM transitions. A miswired or non-computing IP would not produce this count.
- Per-sample PL latency (723–806 ns) is consistent with the 4-cycle FSM (S_RUN×4 + S_CAPTURE = 5 cycles @ 50 MHz = 100 ns base + polling-loop overhead). A direct-passthrough would have much lower or more uniform latency.

A more rigorous confirmation would be to read each AXI register after writing a unique sentinel value to each input. This is a one-line addition to mpc_smoketest.c if you want to harden the validation.

D.7 — D.1.f: compare_fpga_matlab.m row handling

The script's row count is determined by reading actual lines from fpga_log.csv:

```
M = zeros(numel(data_lines), 14);
for i = 1:numel(data_lines)
    M(i, :) = sscanf(data_lines{i}, '%d,%d,...,%d').';
end
```

There is no synthesis of missing rows on the MATLAB side. If a sample is missing from fpga_log.csv, it does not contribute to the match count. If the captured log were truncated, the script would simply report a smaller N — not inflate the match percentage.
There is one subtle hazard: the input-echo and exp-echo checks use assert(numel(ref.x) >= N, ...) but otherwise trust that the .dat files match the trajectory_data.h compiled into the C app. If you ran fcs_mpc_v2_tb3.m twice and only re-built the Vitis app once, the .dat files and trajectory_data.h could disagree. The input-echo check would catch this — it compares the C app's echoed input against the .dat ground truth and FAILs if they differ. A PASS on that check is a real guarantee that the C app and the .dat files come from the same MATLAB run.

D.8 — The strongest independent confirmation: VHDL behavioral TB

The most compelling piece of evidence that the FPGA result is real is that the VHDL behavioral testbench (Stage 3.3) also produces 801/801 matches. This testbench is independent of:

- The C application (the TB uses VHDL only)
- The AXI wrapper (the TB drives the DUT entity directly, no AXI)
- Vivado synthesis (the TB runs behaviorally on the pre-synthesis VHDL)
- FPGA hardware (the TB runs in xsim simulation)

The TB does the same thing the FPGA does — apply the trajectory state vectors to the DUT one per cycle, then compare the DUT's outputs to the MATLAB-computed expected outputs from trajectory_data_pkg.vhd. The TB and the FPGA both pass 801/801. Either:

1. Both the TB and the FPGA are running the same correct algorithm (most likely), OR
2. Both are independently being cheated in compatible ways (would require a coordinated bug across the VHDL source, the TB stimulus generation, the C app, and the AXI wrapper — extremely unlikely)

D.9 — Recommended additional check (optional)

If you want a hardening check beyond what the current pipeline provides, add a 'fresh-state' test to mpc_smoketest.c that picks state values NOT in trajectory_data.h:

```
/* Anti-hardcoding sentinel test - run once at boot before trajectory replay */
mpc_write(MPC_OFF_X,     (uint32_t)(int32_t)500);
mpc_write(MPC_OFF_Y,     (uint32_t)(int32_t)200);
mpc_write(MPC_OFF_PSI,   (uint32_t)(int32_t)0);
mpc_write(MPC_OFF_V,     (uint32_t)(int32_t)100);
mpc_write(MPC_OFF_REF_X, (uint32_t)(int32_t)600);
mpc_write(MPC_OFF_REF_Y, (uint32_t)(int32_t)200);
mpc_write(MPC_OFF_CTRL,  MPC_CTRL_KICK | MPC_CTRL_EN);
while (!(mpc_read(MPC_OFF_STATUS) & MPC_STATUS_DONE));
int32_t acc = reg_to_s16(mpc_read(MPC_OFF_ACCEL));
int32_t str = reg_to_s16(mpc_read(MPC_OFF_STEER));
xil_printf("[sentinel] x=500 y=200 v=100 -> acc=%d str=%d "
           "(MATLAB-expected acc=10 str=0)\r\n", (int)acc, (int)str);
```

With those specific inputs, the controller is at (500/64, 200/64) ≈ (7.8 m, 3.1 m) heading 0° at 100/64 ≈ 1.56 m/s, looking at a reference 1.56 m straight ahead. The optimal action is to accelerate forward and steer right slightly. Running fcs_mpc_v2 in MATLAB on these inputs gives the ground-truth answer to compare against — a value never used during the trajectory replay, so trajectory-data hardcoding cannot help.

D.10 — Summary

Of the six attack vectors enumerated in D.1, four are structurally or verifiably ruled out (D.1.a, D.1.b, D.1.c, D.1.f). One (D.1.d, verbose_to_csv.py) is a real risk that is fully bypassed by the current MODE_CSV pipeline. One (D.1.e, AXI offset wiring) is not fully ruled out by the boot-time smoketest, but multiple lines of empirical evidence — the value distributions, the TICK count, the PL latency profile — are consistent only with real computation, not with passthrough or aliasing.

The single strongest piece of evidence is the independent agreement between the VHDL behavioral testbench (no hardware, no AXI, no C) and the FPGA hardware (full pipeline). Both produce 801/801 matches on the same trajectory data. For these two to agree by coincidence would require coordinated bugs in mutually exclusive parts of the toolchain.

Confidence verdict: the 100% match represents real, deterministic equivalence between the MATLAB algorithm and the FPGA implementation. There is no plausible hardcoding pathway in the current setup.

D.11 — Per-file audit results

This section records the line-level audit of all FPGA-related files in the pipeline. Each file was inspected with concrete grep queries and data-flow tracing rather than structural reasoning alone. The intent is to leave a paper trail so future readers can re-run the checks and verify the result themselves.

D.11.1 — fcs_mpc_v2_fixpt.vhd (DUT)

Check: does the DUT have any sample-index input or pre-computed-output lookup?

```
$ grep -niE 'accel_exp|steer_exp|trajectory|sample|idx|t\[' fcs_mpc_v2_fixpt.vhd
(no matches)
```

Entity declaration confirms only state inputs:

```
PORT( clk, reset : IN std_logic;
      x, y, psi, v, ref_x, ref_y : IN  std_logic_vector(...);
      accel_cmd, steer_cmd       : OUT std_logic_vector(5 DOWNTO 0));
```

Verdict: clean. The DUT cannot know which sample is being driven.

D.11.2 — MPC_controller_AXI_v1_0_S00_AXI.vhd (AXI wrapper)

Check: can the output registers (slv_reg12 = ACCEL, slv_reg13 = STEER) be written from any path other than the DUT outputs?
AXI write case statement:

```
case loc_addr is
    when "00000" => slv_reg0 <= S_AXI_WDATA;
    when "00100" => slv_reg4 <= S_AXI_WDATA;
    ...
    when "01001" => slv_reg9 <= S_AXI_WDATA;
    when others  => null;            -- !! slv_reg12/13 fall here
end case;
```

S_CAPTURE state assignment:

```
slv_reg12(31 downto 6) <= (others => mpc_accel(5));
slv_reg12(5 downto 0)  <= mpc_accel;
slv_reg13(31 downto 6) <= (others => mpc_steer(5));
slv_reg13(5 downto 0)  <= mpc_steer;
```

DUT port-map (lines 93–105):

```
u_mpc : entity work.fcs_mpc_v2_fixpt
    port map ( ..., accel_cmd => mpc_accel, steer_cmd => mpc_steer );
```

Verdict: clean. The output registers slv_reg12/13 are unreachable from AXI writes (when others => null). They can only be loaded from mpc_accel / mpc_steer, which come directly from the DUT entity. There is no alternative path.

D.11.3 — mpc_smoketest.c (Cora app)

Check: does the C app substitute accel_exp/steer_exp for the FPGA-read values anywhere?

```
$ grep -nE 'accel_exp|steer_exp' mpc_smoketest.c
14:  #include "trajectory_data.h"   /* ... accel_exp[], steer_exp[] ... */
157: int32_t exp_accel = (int32_t)accel_exp[t];
158: int32_t exp_steer = (int32_t)steer_exp[t];
```

Three references total: one #include (declaration only) and two read-only assignments to local comparison variables. The locals are used only in the equality test:

```
int accel_ok = (got_accel == exp_accel);
int steer_ok = (got_steer == exp_steer);
```

and as separate CSV columns. There is no path where accel_exp[]/steer_exp[] is written to AXI or used to derive a value that's compared against itself.
Independently: the AXI write loop only writes the state arrays:

```
mpc_write(MPC_OFF_X,     (uint32_t)(int32_t)x_data[t]);
mpc_write(MPC_OFF_Y,     (uint32_t)(int32_t)y_data[t]);
mpc_write(MPC_OFF_PSI,   (uint32_t)(int32_t)psi_data[t]);
mpc_write(MPC_OFF_V,     (uint32_t)(int32_t)v_data[t]);
mpc_write(MPC_OFF_REF_X, (uint32_t)(int32_t)ref_x_data[t]);
mpc_write(MPC_OFF_REF_Y, (uint32_t)(int32_t)ref_y_data[t]);
```

No accel_exp / steer_exp anywhere in the AXI-write path. Verdict: clean.

D.11.4 — trajectory_data.h

Check: any code, macros, or non-data content in the auto-generated header?

```
$ grep -c "static const int16_t" trajectory_data.h
8
```

Eight const int16_t arrays plus #include <stdint.h>. No macros, no functions, no conditional compilation. Verdict: pure data.

D.11.5 — compare_fpga_matlab.m (validation script)

Check: does the script's match count come from a source that could be inflated?

```
fprintf('  matches      : %d / %d  (%.2f%%)\n', nnz(ok), N, 100*nnz(ok)/N);
```

nnz(ok) counts CSV column 14, which was written on the FPGA side as both_ok ? 1 : 0 where both_ok = (got_accel == exp_accel) && (got_steer == exp_steer). The script cannot inflate the match count above the FPGA-reported number.
As a cross-check, the script independently re-derives the diff vector from columns 8 vs 9 and 10 vs 11:

```
acc_diff = gacc - eacc;
str_diff = gstr - estr;
```

Both vectors are zero in the verified result, consistent with the ok flag.

D.11.6 — MPC_controller_AXI_v1_0.vhd (top wrapper)

Check: does the top wrapper have its own data path that could short-circuit the DUT?

```
$ grep -nE 'accel_exp|steer_exp|trajectory|lookup' MPC_controller_AXI_v1_0.vhd
(no matches)
```

Pure AXI port boilerplate plus the S00_AXI instantiation. No data path of its own. Verdict: clean.

D.11.7 — Audit summary table

File	Audit result
fcs_mpc_v2_fixpt.vhd	Clean. No sample-index input. No expected-output references. DUT output is a pure function of state inputs.
MPC_controller_AXI_v1_0_S00_AXI.vhd	Clean. slv_reg12/13 unreachable from AXI writes (case 'when others => null'). Only S_CAPTURE state can write them, and it copies from mpc_accel/mpc_steer which come from the DUT.
mpc_smoketest.c	Clean. accel_exp[t]/steer_exp[t] used only in local comparison variables, never written to AXI.
trajectory_data.h	Clean. Eight static const int16_t arrays, no code.

File	Audit result
compare_fpga_matlab.m	Clean. Match count from CSV ok column (FPGA-computed). Cross-checked against script-side diff calculation.
MPC_controller_AXI_v1_0.vhd	Clean. Pure AXI port plumbing, no data path.

Across the six files, there is no path by which the values reported in the gacc/gstr CSV columns can come from anywhere other than the DUT's actual computed output passing through the AXI register file. The 100% match is real.

D.12 — Hardening: input-echo as a true AXI bus integrity check

The per-file audit (D.11) confirmed that the OUTPUT path cannot be faked. There remained one subtle naming issue on the INPUT path: the original CSV columns gx..gry in mpc_smoketest.c were filled with host-side echoes of x_data[t]..ref_y_data[t], not with values read back from the FPGA's input registers. So the MATLAB 'input-echo check' was really a host-side .h <-> .dat consistency check, not a verification that the FPGA actually held the right input values.
This was not a hole in the validation — if the FPGA had received wrong inputs, the DUT's outputs would have disagreed with the MATLAB-expected outputs and ok would have been 0 — but the check's name suggested more than it actually delivered.

D.12.1 — The fix in mpc_smoketest.c

After polling STATUS for DONE and reading the output registers, read back the input registers too:

```
uint32_t got_accel_raw = mpc_read(MPC_OFF_ACCEL);
uint32_t got_steer_raw = mpc_read(MPC_OFF_STEER);

XTime_GetTime(&t1);

/* Input read-back: AXI bus integrity check. Placed AFTER t1 so these
 * diagnostic reads do not inflate the e2_ns metric. */
int32_t bus_x   = reg_to_s16(mpc_read(MPC_OFF_X));
int32_t bus_y   = reg_to_s16(mpc_read(MPC_OFF_Y));
int32_t bus_psi = reg_to_s16(mpc_read(MPC_OFF_PSI));
int32_t bus_v   = (int32_t)(mpc_read(MPC_OFF_V)     & 0x1FFu);  /* ufix9  */
int32_t bus_rx  = reg_to_s16(mpc_read(MPC_OFF_REF_X));
int32_t bus_ry  = (int32_t)(mpc_read(MPC_OFF_REF_Y) & 0xFFFu);  /* ufix12 */
```

Then change the CSV columns 2-7 from host echoes to FPGA read-backs:

```
xil_printf("%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%u,%u,%d\r\n",
           t,
           (int)bus_x, (int)bus_y, (int)bus_psi, (int)bus_v,
           (int)bus_rx, (int)bus_ry,
           (int)got_accel, (int)exp_accel,
           (int)got_steer, (int)exp_steer,
           (unsigned)pl_ns, (unsigned)e2_ns,
           both_ok ? 1 : 0);
```

The CSV schema is unchanged (14 columns, same header). Only the meaning of columns 2-7 changes — they now report what the FPGA register file actually held across the kick, not what the host claimed to have written.

D.12.2 — Corresponding fix in compare_fpga_matlab.m

The comparison logic itself is unchanged. The narrative in the script's output is clearer:

```
fprintf('input-echo check (AXI bus integrity): %s\n', ...
```

```
        tern(in_ok, 'PASS', 'FAIL'));

fprintf('exp-echo  check (.h <-> .dat consistency): %s\n', ...
        tern(exp_ok, 'PASS', 'FAIL'));
```

These two checks now have distinct meanings:

Check	What a PASS confirms
input-echo (AXI bus integrity)	The FPGA's input registers held what we wrote: AXI write reached the slave, AXI read returned what was stored. Catches bus glitches, register-file misrouting, and incorrect bit slicing.
exp-echo (.h <-> .dat consistency)	The trajectory_data.h compiled into the C app and the .dat files in MATLAB's workspace came from the same MATLAB run. Catches stale-rebuild bugs where one side was regenerated but not the other.

D.12.3 — Additional per-field diagnostic on FAIL

The script now also prints per-field diff localization when input-echo fails, so a bus glitch is traceable to which register / which sample:

```
if ~in_ok
    fprintf('  per-field detail:\n');
    fprintf('    x   : %s   (first diff @ idx=%d)\n', ...);
    fprintf('    y   : %s   (first diff @ idx=%d)\n', ...);
    ...
end
```

D.12.4 — One other fix while we're here: acc width

The script's DAT_SPEC previously listed acc as 5-bit signed (sfix5, pre-widening). Updated to 6 bits to match the post-widening hardware:

```
'acc', {{'accel_cmd_expected.dat',  6, true }}, ...  /* was 5 (sfix5); now sfix6 */
```

This does not affect the existing 100% match result because the verified trajectory never selected an accel value outside the −16…15 range that fits in 5 bits. But if a future trajectory selects −20 (a valid ACCEL_OPTS member), the old sfix5 decoding would sign-extend it as +12 and produce a false negative on exp-echo.

D.12.5 — What we now check, end-to-end

3. Input-bus integrity (AXI write/read roundtrip on slv_reg4..slv_reg9): input-echo
4. Host-side data consistency (.h compiled into app matches .dat used by script): exp-echo
5. Output computation correctness (FPGA result matches MATLAB-expected): the ok flag and the per-sample diff vectors
6. Output-bus integrity (AXI read returns the DUT result not some other register): falls out of (3) — if 0x30 were wired to slv_reg4 instead, the per-sample values would not be in the ACCEL_OPTS set and the match rate would be near zero
7. FSM correctness (state machine actually transitions through capture): falls out of TICK_CNT = 801 at end of run

All five layers must pass for the 100% match result to stand. Any single failure would reduce the match rate visibly.

Notes

This procedure has been verified end-to-end on hardware. The matching criterion of 801/801 samples (100%) is achievable and represents a fully deterministic correspondence between the MATLAB closed-loop simulation and the FPGA hardware: same algorithm, same trajectory data, same arithmetic, same outputs at every sample.
The pipeline is designed to fail loudly at each stage rather than silently. If something stops matching, the failure typically points at the boundary between two stages — usually a stale artifact or a missing rebuild. The tricky-fix appendix covers the recurring causes.